

Domain 2

Tool Design & MCP Integration

18% of exam · 5 task statements

2.1 Tool interfaces · 2.2 Structured errors · 2.3 Tool distribution · 2.4 MCP servers · 2.5 Built-in tools

2.1 Designing Effective Tool Interfaces

Descriptions, disambiguation, and splitting

Why tool descriptions are critical

Tool descriptions are the **primary mechanism** the model uses to decide which tool to call. Claude cannot read source code or infer purpose — it only sees the name, description, and schema. Minimal or overlapping descriptions lead to unreliable tool selection and misrouting.

**KEY
CONCEPT**

Every tool description must answer 4 questions: (1) what does this tool do, (2) what inputs does it expect, (3) what does it return, (4) when should you use it instead of similar alternatives.

Required elements in every tool description

Element	Why it matters	Example
Purpose	Tells Claude what the tool does	Searches the product catalogue by SKU or name
Input format	Prevents malformed calls	Accepts ISO 8601 date strings only, not Unix timestamps
Output description	Sets expectations for downstream parsing	Returns a JSON array of up to 10 matching products
Edge cases	Prevents misuse on boundary inputs	Returns empty array (not error) when no products match
When NOT to use	Disambiguates from functionally similar tools	Use search_orders for order lookup, not this tool

The 2 root causes of misrouting

Overlapping descriptions

- analyze_content vs analyze_document with near-identical descriptions
- Model picks arbitrarily between them — no reliable basis for selection
- Fix: rename + rewrite to eliminate functional overlap
- e.g. rename analyze_content to extract_web_results with a web-specific description

Keyword-sensitive system prompts

- System prompt says 'always analyze documents thoroughly'
- Keyword 'analyze' accidentally triggers the wrong tool
- Model obeys prompt keywords over description logic
- Fix: audit system prompt for accidental keyword associations with tool names

Splitting generic tools into purpose-specific ones

A generic analyze_document tool does too many things — the model doesn't know which behaviour to invoke. Split it into purpose-specific tools with defined input/output contracts:

extract_data_points Pulls structured fields from documents — dates, amounts, names, IDs	summarize_content Generates a prose summary of the document's main arguments or findings	verify_claim_against_source Checks a specific assertion against document evidence and returns a verdict
---	--	---

EXAM TRAP — TOOL INTERFACES

Model consistently calls the wrong tool even though both tools work correctly. The answer is never 'improve the model' or 'set tool_choice: any'. Fix: rewrite/rename tool descriptions to eliminate overlap, or audit the system prompt for keyword interference.

2.2 Structured Error Responses for MCP Tools

isError flag, error categories, and recovery patterns

The isError flag pattern

MCP tools communicate failures back to the agent using the `isError` flag in the tool result. Without structured error metadata, the agent cannot decide whether to retry, escalate, or report a user-friendly message. A generic "Operation failed" response is an anti-pattern — it leaves the agent unable to recover intelligently.

KEY CONCEPT

Structured error response must include: `errorCategory` (transient/validation/business/permission) + `isRetryable` boolean + human-readable description. This gives the agent everything it needs to make the right recovery decision.

4 error categories — know all of them

Category	Cause	Retryable?	Agent response
Transient	Timeout, service unavailable, network error	YES	Retry with exponential backoff
Validation	Invalid input format, missing required field	NO — fix input first	Correct the input, then retry with valid data
Business	Policy violation (e.g. refund exceeds limit)	NO	Escalate to human or explain constraint to user
Permission	Insufficient access rights, auth failure	NO	Request different credentials or escalate

Local recovery vs coordinator escalation

Handle locally (subagent) - Transient failures — retry with backoff before escalating - Errors the subagent can resolve itself without coordinator involvement - Don't escalate until all local recovery options are exhausted	Propagate to coordinator - Errors that cannot be resolved locally after retries - Must include: partial results + what was attempted + error details - Coordinator then decides: retry differently, skip, or escalate to human
---	---

Empty result vs access failure — critical distinction

An **access failure** (couldn't connect, permission denied) needs a retry decision — set `isError: true`. A **valid empty result** (query succeeded, zero matches found) is a successful response — set `isError: false`, return an empty array. Treating empty results as errors causes the agent to retry unnecessarily, wasting calls and confusing the reasoning loop.

EXAM TRAP — STRUCTURED ERRORS

A search tool returns a generic error for 'no results found'. The agent keeps retrying. Root cause: the tool didn't distinguish between 'no results' (success, `isError: false`, empty array) and 'search failed' (error, `isError: true`). Fix: always return structured empty results for zero-match queries — never a generic failure.

2.3 Tool Distribution & tool_choice Configuration

Scoping, least privilege, and tool_choice options

Too many tools degrades reliability

Giving an agent 18 tools instead of 4–5 degrades tool selection reliability by increasing decision complexity. The model's attention is spread across too many options, and agents with tools outside their specialisation tend to misuse them. A synthesis agent with access to web_search will attempt searches even when it shouldn't.

**KEY
CONCEPT**

Principle of least privilege for tools: each agent gets only the tools needed for its role. Cross-role tools may be allowed sparingly for high-frequency needs, but the default is strict scoping.

tool_choice configuration options

Value	Behaviour	When to use
"auto"	Claude decides whether to call a tool or respond with text	Default — the right choice for most use cases
"any"	Claude must call some tool (but chooses which one)	When you need guaranteed tool use, e.g. structured output extraction
{"type":"tool","name":"X"}	Claude must call this specific named tool	When you need a specific tool called deterministically every time
"none"	Claude cannot call any tools — text response only	When you want a pure reasoning or summary response with no tool calls

**EXAM TRAP —
TOOL_CHOICE**

Setting tool_choice: 'any' to fix misrouting between tools. It guarantees a tool call but Claude still picks WHICH tool — so ambiguous descriptions still cause the wrong tool to be selected. The fix is always tool description quality, not tool_choice configuration.

Tool scoping by agent role — example

Agent role	Gets access to	Does NOT get
Web search agent	search_web, fetch_url	write_file, send_email, process_payment
Document analysis	read_file, extract_text, parse_pdf	web_search, send_email, process_payment
Synthesis agent	write_report, format_output	web_search, read_file, process_payment
Billing agent	process_payment, lookup_invoice	web_search, read_file, write_report

2.4 Integrating MCP Servers

Primitives, transports, scopes, and configuration

What is MCP?

The **Model Context Protocol** is Anthropic's open standard for connecting Claude to external tools, data sources, and services. It provides a standardised interface so agents can interact with external systems without writing custom integration code for each one.

MCP primitives — the 3 you must know cold

Tools

Functions the model can call to take actions or retrieve data. Defined with name, description, and input_schema. Claude decides when to call them based on their descriptions.

Resources

Read-only data sources the model can access (files, DB records, API responses). Expose content catalogs as resources so agents can see available data without needing exploratory tool calls.

Prompts

Pre-defined prompt templates the server exposes. Reusable instruction sets injected into conversations. Useful for consistent task framing across multiple agent calls.

Transport mechanisms — stdio vs HTTP/SSE

stdio (standard I/O) • MCP server runs as a local subprocess • Communication via stdin/stdout pipes • Best for: local tools, CLI integrations, Claude Code • Simple to set up, low latency for local use • Cannot serve multiple clients simultaneously	HTTP/SSE (server-sent events) • MCP server runs as a remote HTTP service • Communication via HTTP + server-sent events • Best for: shared team servers, cloud services, production • Supports authentication and multiple simultaneous clients • Requires network infrastructure but scales better
--	---

MCP configuration scopes

Scope	File location	Use for	Shared with team?
Project-scoped	.mcp.json in project root	Shared team MCP servers — checked into version control	YES
User-scoped	~/.claude.json	Personal or experimental MCP servers not ready for sharing	NO — user only

KEY CONCEPT

Environment variable expansion: use \${ENV_VAR} syntax in .mcp.json for auth tokens and secrets. Never hardcode credentials in config files that get committed to version control.

Community servers vs custom servers

Community MCP servers (prefer these) • Use for standard integrations: Jira, GitHub, Slack, Notion, etc. • Already built, tested, and maintained by the community • Faster to integrate than building from scratch • Example: use the community Jira MCP server — don't build your own	Custom MCP servers (only when needed) • Reserve for team-specific or proprietary workflows • Internal APIs with no existing community server • Custom business logic that can't be handled generically • More work to build and maintain — justify the investment
--	--

Enhancing MCP descriptions to prevent built-in tool fallback

By default, Claude may prefer built-in tools (like Grep) over MCP tools for familiar-sounding tasks, because built-in tools have well-known descriptions. If an MCP tool is more capable, its description must explicitly explain its superior capabilities — otherwise Claude falls back to the built-in.

Example: an MCP semantic search tool with a vague description will lose to Grep. Enhance the description to explain it does semantic matching (not keyword matching) and returns ranked results with relevance scores.

EXAM TRAP — MCP SCOPE	A developer adds an MCP server to ~/.claude.json and wonders why teammates can't use it. Personal scope (~/.claude.json) is user-only. For shared team access, the server must be defined in project-scoped .mcp.json.
EXAM TRAP — CREDENTIALS	Auth tokens hardcoded in .mcp.json — these get committed to version control and become a security risk. Always use \${ENV_VAR} expansion for secrets. The value is resolved from the environment at runtime, not stored in the file.

2.5 Selecting Built-in Tools Effectively

Read, Write, Edit, Bash, Grep, Glob

Claude Code built-in tools — complete reference

Tool	What it does	Key constraint / when to use
Read	Read file contents from the filesystem	Read-only — cannot modify files. Use before Edit to inspect current state.
Write	Create or overwrite a file entirely	Replaces the WHOLE file. Use only for new files or full replacements. Risk: loses existing content not included in the new write.

Edit	Make targeted changes to specific lines in a file	PREFERRED over Write for incremental changes. Safer, more precise, preserves unchanged content.
Bash	Execute shell commands	Most powerful and most dangerous. Scope permissions carefully — restrict by allowed directories and whitelisted commands.
Grep	Search file contents by regex pattern	Keyword / pattern matching only — NOT semantic search. Use for exact strings, regex, or import scanning.
Glob	Find files matching a path pattern	Returns file PATHS only, not contents. Follow with Read to get file contents.

KEY CONCEPT

Edit vs Write: use Edit for targeted line-level changes. Use Write only when creating a new file or replacing the entire content. Write on an existing file discards all content not explicitly included — a common source of accidental data loss in agentic workflows.

Bash tool — power and risk

Bash can execute anything the shell can — install packages, call APIs, modify system files, run tests, deploy code. In production agentic systems, Bash permissions must be scoped:

Restrict by path - Limit which directories can be read and written - Prevent access outside the project directory - Configure via allowedDirectories in Claude Code settings	Restrict by command - Whitelist specific safe commands (e.g. npm test, pytest, git status) - Block destructive commands (rm -rf, curl to external hosts) - Configure via Claude Code hooks or CLAUDE.md rules
---	--

Grep vs MCP semantic search — when to use which

Grep (exact/pattern matching) - Finding exact strings or regex patterns in files "Find all TODO comments in the codebase" "Find all files that import React" - Fast, deterministic, no AI involved	MCP semantic search (conceptual matching) - Finding conceptually related content without exact keywords "Find functions related to authentication" "Find documentation about error handling patterns" - Returns ranked results by relevance, not just pattern hits
---	---

All Domain 2 Exam Traps		Memorise before exam day
EXAM TRAP — TOOL INTERFACES	Model consistently calls the wrong tool — answer is never 'improve the model'. Fix: rewrite tool descriptions to eliminate overlap, or audit system prompt for keyword interference.	
EXAM TRAP — TOOL INTERFACES	Setting tool_choice: 'any' to fix misrouting — it guarantees A tool call but Claude still picks WHICH tool. Description quality is the fix, not tool_choice.	
EXAM TRAP — TOOL INTERFACES	A generic analyze_document tool causes unreliable behaviour — split it into purpose-specific tools (extract_data_points, summarize_content, verify_claim_against_source) with distinct, non-overlapping descriptions.	
EXAM TRAP — STRUCTURED ERRORS	Using a generic 'Operation failed' error message — the agent cannot decide whether to retry, skip, or escalate. Always return errorCategory + isRetryable + human-readable description.	
EXAM TRAP — STRUCTURED ERRORS	Returning isError: true for 'no results found' — a zero-match query is a successful response (isError: false, empty array). Only genuine failures should set isError: true.	
EXAM TRAP — STRUCTURED ERRORS	Subagent escalates every error to the coordinator without attempting local recovery first. Transient errors should be retried locally. Only propagate errors that cannot be resolved, and include partial results + what was attempted.	

EXAM TRAP — TOOL DISTRIBUTION	Giving agents access to all available tools for 'maximum flexibility' — too many tools degrades selection reliability. Scope each agent to only the tools its role requires.
EXAM TRAP — MCP SCOPE	Adding an MCP server to ~/.claude.json and expecting teammates to see it — personal scope is user-only. Shared servers must go in project-scoped .mcp.json.
EXAM TRAP — MCP CREDENTIALS	Hardcoding auth tokens in .mcp.json — these get committed to version control. Always use \${ENV_VAR} expansion for secrets.
EXAM TRAP — MCP DESCRIPTION	Claude prefers Grep over a better MCP semantic search tool — because the MCP description is vague. Enhance the MCP tool description to explicitly explain its superior capabilities.
EXAM TRAP — BUILT-IN TOOLS	Using Write to update a single line in a file — Write replaces the ENTIRE file. Use Edit for targeted changes. Write on an existing file discards all content not in the new version.
EXAM TRAP — COMMUNITY VS CUSTOM	Building a custom MCP server for Jira, GitHub, or Slack — community servers already exist for standard integrations. Reserve custom servers for proprietary team workflows with no existing community option.

Domain 2 complete — 18% of the CCA-F exam. Next: Domain 3 · Claude Code Configuration & Workflows (20%).